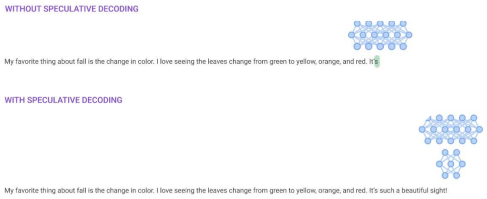
Google Research

Home > Blog >

Looking back at speculative decoding



December 6, 2024

Yaniv Leviathan, Distinguished Engineer, Matan Kalman, Software Engineer, and Yossi Matias, Vice President & Head, Google Research

Speculative decoding has proven to be an effective technique for faster and cheaper inference from LLMs without compromising quality. It has also proven to be an effective paradigm for a range of optimization techniques.

QUICK LINKS

Paper

Share

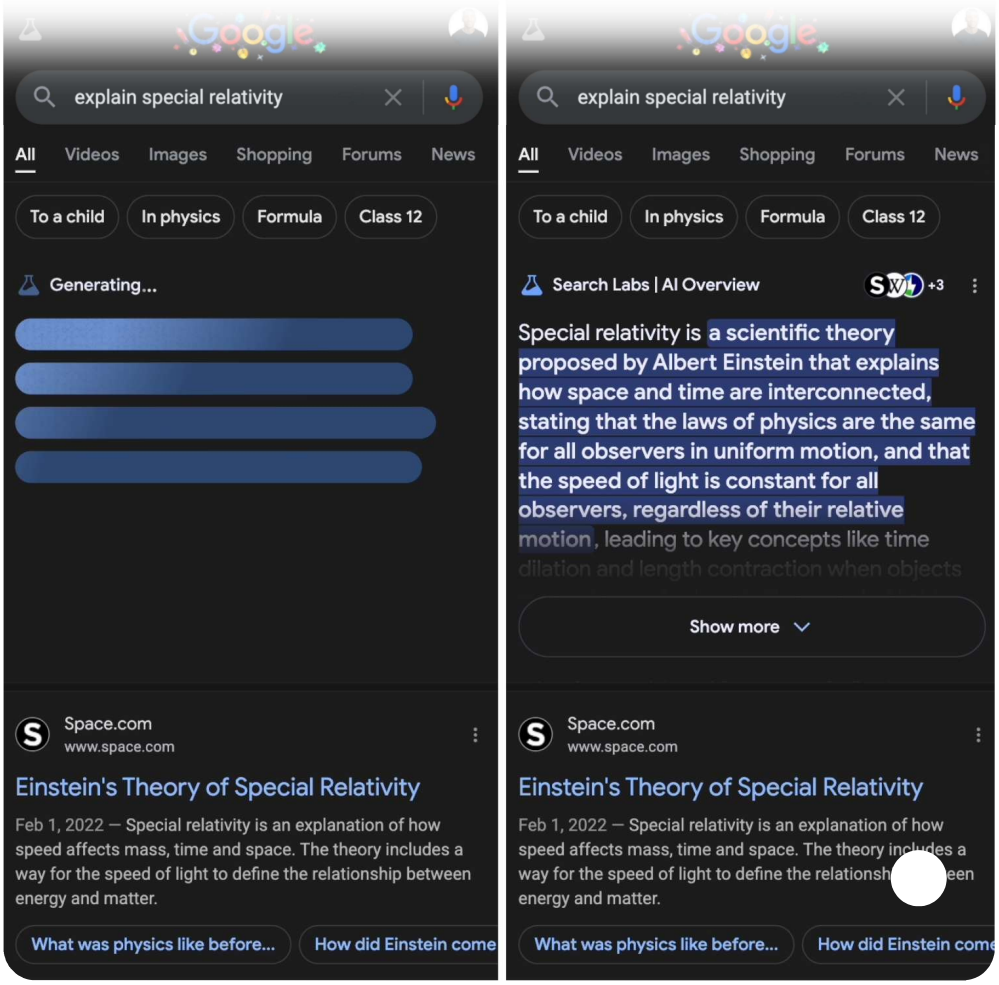
Large language models (LLMs) are at the center of the recent rapid progress in artificial intelligence (AI). While groundbreaking, a challenge for user-facing products is that due to their size, these large models are slow at inference (i.e., output generation), which may result in an undesirably slow user experience.

Research

autoregressive models by computing several tokens in parallel, without affecting output quality; in fact, the method guarantees an identical output distribution. *Producing results faster with the same hardware also means that fewer machines are needed for serving the same amount of traffic, which translates yet again to a reduction in the energy costs of serving the same model.*

Today, we look back at how the method works, explore its inspiration from speculative execution, and highlight some new insightful developments. In our original paper, we demonstrated the efficacy of this approach through application to translation and summarization tasks, where we saw *~2x–3x improvements*. Since then, we have applied speculative decoding in a number of Google products, where we see remarkable speed-ups in inference, while maintaining the same quality of responses. We have also seen speculative decoding adopted throughout the industry, and have witnessed many insightful and effective applications and ideas using the speculative decoding paradigm.

Research



Speculative decoding enables AI Overviews in Google Search to produce results faster than before, while maintaining the same quality of responses. Large scale products, like AI Overviews, are continuously optimized with new techniques, yet speculative decoding remains a significant part of the optimizations.

Background

An LLM generates its output one token at a time, where a token is usually a word or a part of a word. For example, with a common tokenizer, the sentence “One small step for man, one giant leap for mankind” is composed of 12 tokens. That means that to generate this sentence the LLM must run 12 times. Each such run is called a decoding step.

Research

order of a tera-byte of data for each word it produces!

Typically, the goal is for LLMs to generate many words, such as a conversational response or a summary of a document, and since each token depends on the ones previously produced, they must be generated one by one, reading all of the model's weights again and again. Two key observations motivate our speculative decoding method.

Observation 1: Some tokens are easier to generate than others

Not all tokens are alike: some are harder and some are easier to generate. Consider the following text:

`What is the square root of 7? The square root of **7** is ***2.646***.`

Generating the emphasized token "***7***" is relatively easy; for example, we can notice that the previous tokens "square root of" happened before, and just copy the following token. Generating the tokens "***2.646***" is harder; the model would need to either compute or remember the answer.

This observation suggests that the large models are better due to better performance in difficult cases (e.g. "***2.646***"), but that in the numerous easy cases (e.g., "***7***"), small models might provide reasonable approximations for the large models.

Observation 2: The bottleneck for LLM inference is usually memory

Machine learning hardware varieties, TPUs and GPUs, are highly parallel machines, usually capable of *hundreds of trillions* of operations per second, while their memory bandwidth is usually around just *trillions* of bytes per second — a couple of

Research

In contrast, the Transformer architecture that underlies modern LLMs usually performs only a few operations for every byte read during inference, meaning that there are ample spare computational resources available when generating outputs from LLMs on modern hardware.

Hardware can do	Transformers need
~100s–1000s operations/byte read	**~10** operations/byte read

Speculative execution

Based on the expectation that additional parallel computational resources are available while tokens are computed serially, our method aims to increase concurrency by computing several tokens in parallel. The approach is inspired by speculative execution, an optimization technique whereby *a task is performed before or in parallel with the process of verifying whether it is actually needed*, resulting in increased concurrency. A well-known example of speculative execution is branch prediction in modern pipelined CPUs.

For speculative execution to be effective, we need an efficient mechanism that can suggest tasks to execute that are likely to be needed. More generally, consider this abstract setting for speculative execution, with the assumption that $f(X)$ and $g(Y)$ are lengthy operations:

$Y = f(X)$

$Z = g(Y)$

The slow function $f(X)$ computes Y, which is the input to the slow function $g(Y)$. In the setting above, $f(X)$ and $g(Y)$ are the same function. Without speculative execution, we'd need to evaluate these serially. Speculative execution suggests that given any fast approximation function $f^*(X)$, we can evaluate the first slow operation $f(X)$ in parallel to evaluating $g(f^*(X))$. Once $f(X)$ finishes and we obtain the correct value of Y, we can check if the output of the fast approximation $f^*(X)$ was Y as well, in which case we managed to increase parallelization. If $f^*(X)$ output a

Research

concurrency. We are guaranteed identical outputs either way.

Speculative sampling

We proposed a generalization of speculative execution to the stochastic setting, i.e., where a task *needs to be executed with some probability*. We call this generalization *speculative sampling*. Consider the following setup, identical to that above, with the exception that $f(X)$ now outputs a probability distribution from which we *sample* the input to function g, Y:

$Y \sim f(X)$

$Z = g(Y)$

Similar to above, given any fast approximation $f^*(X)$, this time outputting a probability distribution, speculative sampling allows us to execute $f(X)$ in parallel to the execution of $g(sample(f^*(X))$. We could use standard speculative execution, and discard the calculation in case the samples don't match, but this would be inefficient. Indeed, consider an example where $f(X)$ and $f^*(X)$ always output a uniform probability distribution from 1 to 100. Speculative execution would only accept f^*'s guess once every 100 times. This is clearly inefficient — f and f^* are the same function and we can always accept the sample from f^*! Instead, speculative sampling offers a way to accept or discard f^*'s guesses probabilistically, depending on $f(X)$ and $f^*(X)$, guaranteeing optimality as well as an identical output distribution. Speculative sampling could be useful in other settings, for example, in reinforcement learning or physics simulations (see paper for details).

Speculative decoding

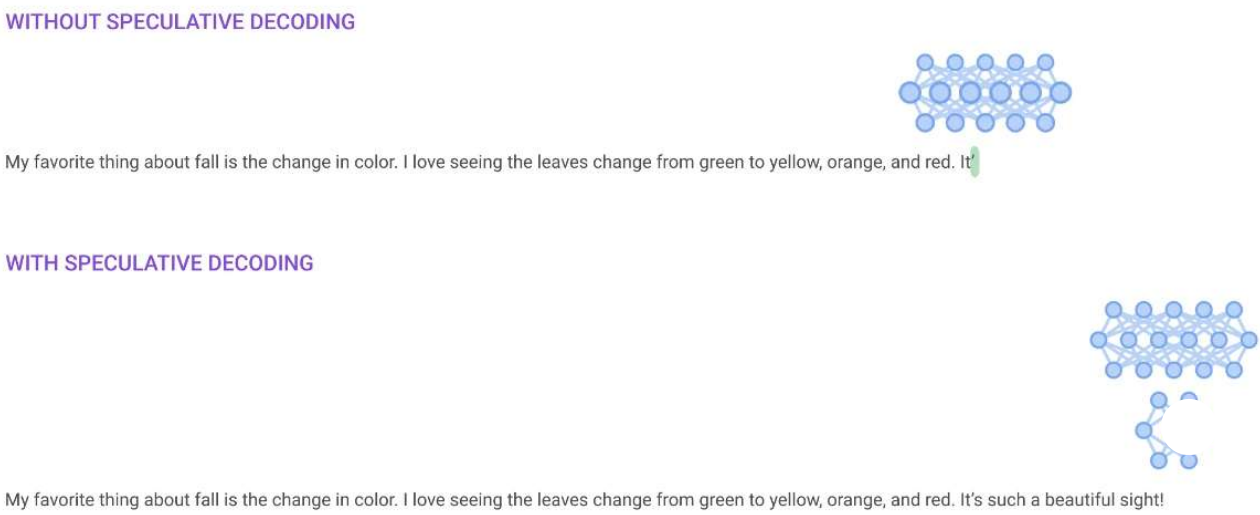
LLMs don't produce a single next token, but rather a probability distribution from which we sample the next token (for example, following the text "The most well known movie director is", an LLM might produce the token "Steven" with 70% chance and the token "Quentin" with 30% chance). This means that a direct

Research

samples come from exactly the same probability distribution as those produced by naïve decoding. Note that in the special case of greedy decoding, where we always sample the single most probable token, speculative execution can be applied effectively to LLM inference, as was shown in a precursor to our work.

Speculative decoding is the application of speculative sampling to inference from autoregressive models, like transformers. In this case, both $f(X)$ and $g(Y)$ would be the same function, taking as input a sequence, and outputting a distribution for the sequence extended by one token. Speculative decoding thus allows us to efficiently calculate a token and the tokens following it, in parallel, while maintaining an identical distribution (note that speculative decoding can parallelize the generation of more than two tokens, see the paper).

All that remains in order to apply speculative decoding is a fast approximation of the decoding function. Observation 1 above suggests that a small model might do well on many of the easier tokens. Indeed, in the paper we showed that using existing off-the-shelf smaller models or simple heuristics works well in practice. For example, when applying speculative decoding to accelerate an 11B parameter T5-XXL model for a translation task, and using a smaller 60M parameter T5-small as the guessing mechanism, we get ~3x improvement in speed.



An animation, demonstrating the speculative decoding algorithm in comparison to standard decoding. The text is generated by a large GPT-like transformer decoder. In the case of

Research

Towards more efficient AI

We have seen speculative decoding adopted across the industry, with some remarkable reported performance gains. This wide adoption was accelerated by the numerous insightful and effective techniques using the speculative decoding paradigm, often in tandem with other novel methods. For example, showing effectiveness for large scale models in a distributed setup, using several draft guesses instead of one, distilling the knowledge from the target model into the draft model, letting the draft model use part of the target model, using a single model for both draft and target, or verifying all draft tokens together. The method was also applied to domains such as image and speech generation.

With the growing usage of LLMs, the need for more efficient inference becomes increasingly more important. We are looking forward to seeing additional ideas, utilizing the speculative decoding and other existing paradigms, as well as entirely new approaches.

Acknowledgements

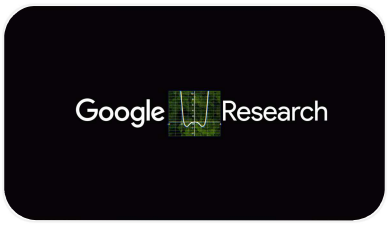
This work is the result of a close collaboration with Asaf Aharoni, Avinatan Hassidim, and Danny Vainstein. In addition, we'd like to extend a huge thank you for reviews, help, insightful discussions, valuable feedback and support to YaGuang Li, Blake Hechtman, Tao Wang, Toby Boyd, Nathan Lintz, Phil Chen, Nir Shabat, Jayant Madhavan, Aliaksei Severyn, Jakub Adamek, Jonathan Mallinson, Zhifeng Chen, Yoel Drori, Mariano Schain, Charlie Chen, Noam Velan, Nitish Kulkarni, Sidharth Mudgal, Sasha Goldshtein, Nadav Sherman, Pilar Manchon, Fernando Pereira, Eyal Segalis, Eyal Molad, Dani Valevski, Daniel Lumen, Valerie Nygaard, Steve Baker, Srinivasan (Cheenu) Venkatachary, Hema Budaraju, Ziteng Sun, Ananda Theertha Suresh, Elizabeth Hamon Reid, Jeff Dean, Prabhakar Raghavan, James Manyika, and teams in Google Research, Google Deepmind, and Google Search.

Research

Generative AI

Machine Intelligence

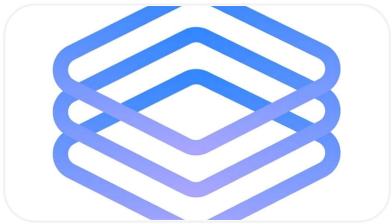
Other posts of interest



OCTOBER 14, 2026

Research@ 2026: 10 ways we're amplifying human ingenuity

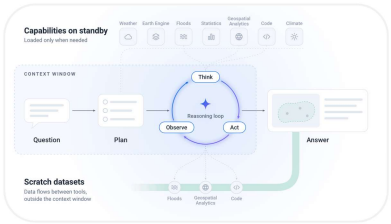
General Science · Generative AI · Health & Bioscience



OCTOBER 14, 2026

Analyzing technical documents with Deep Review

General Science · Machine Intelligence · Natural Language Processing



OCTOBER 14, 2026

Putting Earth AI to work: Geospatial agents for our planet's biggest challenges

Earth AI · General Science · Machine Intelligence

Follow us

Explore our other initiatives

Google AI

Discover how Google AI is committed to enriching knowledge and solving complex challenges

Products

Google Cloud

High-performance infrastructure for cloud computing, data analytics & machine learning

Overview

Research

Responsibility

Societal Impact

About

Pricing

Resources

Google DeepMind

Our mission is to build AI responsibly to benefit humanity

Models

Research

Science

About

Google Labs

Explore the future of AI responsibly with Google Labs

About

Experiments

Stay connected

Google About Google Google Products Privacy Terms